

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 5
з курсу “Апаратне забезпечення вбудованих систем”
на тему: “Передача даних через UART”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Паранчук Р.Я.

Львів – 2025

Завдання: опанувати надсилання та отримання даних через інтерфейс UART. Запустити один із варіантів 4.1 – 4.F, заданий викладачем. З'єднавши виводи TX та RX провідником із захисним резистором, протестувати роботу системи у режимі Loopback. Від'єднати провідник і дати плату викладачу для перевірки на «тестувальному стенді»

Інструменти: Visual Studio Code з розширенням ESP-IDF та мікроконтроллером ESP32 C3.

4.1. Запит: п'ятизначне випадкове число від 00000 до 99999, при потребі доповнене нулями зліва щоб завжди було 5 цифр! Надсилати кожні 5 секунд. Наприклад: (1) _ 55223\n

Код:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#include "driver/uart.h"
#include "esp_system.h"
#include "esp_log.h"

static const char *TAG = "uart_lab";

#define UART_PORT_NUM UART_NUM_1
#define UART_TX_PIN 4
#define UART_RX_PIN 5
#define UART_BAUD_RATE 1200

// Buffers
#define RX_BUF_SIZE 1024
#define LINE_MAX_LEN 128

// Variant id and timing for 4.1
static const char *VARIANT_PREFIX = "(1) ";
static const TickType_t TX_PERIOD_TICKS = pdMS_TO_TICKS(5000); // 5 seconds

// Forward
static void uart_tx_task(void *arg);
static void uart_rx_task(void *arg);

void app_main(void)
{
    // UART configuration per Lab specification: 1200, 8 data bits, EVEN parity, 2 stop bits
    uart_config_t uart_config = {
        .baud_rate = UART_BAUD_RATE,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_EVEN,
```

```

        .stop_bits = UART_STOP_BITS_2,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
        .rx_flow_ctrl_thresh = 0};

// Configure UART parameters
ESP_ERROR_CHECK(uart_driver_install(UART_PORT_NUM, RX_BUF_SIZE * 2, 0, 0, NULL,
0));
ESP_ERROR_CHECK(uart_param_config(UART_PORT_NUM, &uart_config));
ESP_ERROR_CHECK(uart_set_pin(UART_PORT_NUM, UART_TX_PIN, UART_RX_PIN,
UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE));

    ESP_LOGI(TAG, "UART%d configured TX=%d RX=%d, %dbps, 8,E,2", UART_PORT_NUM,
UART_TX_PIN, UART_RX_PIN, UART_BAUD_RATE);

// Create tasks
xTaskCreate(uart_tx_task, "uart_tx_task", 4096, NULL, 5, NULL);
xTaskCreate(uart_rx_task, "uart_rx_task", 4096, NULL, 6, NULL);
}

/**
 * Helper: generate 5-digit number as string with leading zeros
 */
static void gen_5digit_str(char *out, size_t out_sz)
{
    uint32_t r = esp_random() % 100000; // 0..99999
    snprintf(out, out_sz, "%05u", (unsigned)r);
}

/**
 * Helper: reverse digits (string) in place
 */
static void reverse_str(char *s)
{
    size_t i = 0, j = strlen(s);
    if (j == 0)
        return;
    j--;
    while (i < j)
    {
        char t = s[i];
        s[i] = s[j];
        s[j] = t;
        i++;
        j--;
    }
}

/**
 * TX task: every 5 seconds send "(1) 12345\n"
 */
static void uart_tx_task(void *arg)
{
    char numstr[8];

```

```

    char req[32];
    while (1)
    {
        gen_5digit_str(numstr, sizeof(numstr));
        // Format per spec: (1)_XXXXX\n
        snprintf(req, sizeof(req), "%s%s\n", VARIANT_PREFIX, numstr);
        int len = strlen(req);
        int written = uart_write_bytes(UART_PORT_NUM, req, len);
        ESP_LOGI(TAG, "TX: %s (wrote %d bytes)", req, written);
        vTaskDelay(TX_PERIOD_TICKS);
    }
}

/**
 * RX task: read bytes, accumulate until '\n', then process lines.
 * If line starts with "(1) " -> handle variant 4.1: reverse digits and send
response WITHOUT variant prefix.
 * Otherwise ignore.
 */
static void uart_rx_task(void *arg)
{
    uint8_t buf[128];
    char line[LINE_MAX_LEN];
    size_t line_pos = 0;

    while (1)
    {
        int rx_bytes = uart_read_bytes(UART_PORT_NUM, buf, sizeof(buf),
pdMS_TO_TICKS(200));
        if (rx_bytes > 0)
        {
            for (int i = 0; i < rx_bytes; ++i)
            {
                char c = (char)buf[i];
                if (c == '\r')
                    continue; // ignore CR
                if (line_pos < LINE_MAX_LEN - 1)
                {
                    line[line_pos++] = c;
                }
                // End of line?
                if (c == '\n')
                {
                    line[line_pos] = '\0';
                    // Trim trailing newline for processing
                    size_t len = line_pos;
                    if (len > 0 && line[len - 1] == '\n')
                        line[len - 1] = '\0';
                    ESP_LOGI(TAG, "RX line: \"%s\"", line);

                    // Check prefix "(1) "
                    if (strncmp(line, VARIANT_PREFIX, strlen(VARIANT_PREFIX)) == 0)
                    {

```

```

char *payload = line + strlen(VARIANT_PREFIX); // expected
"12345"

// For safety, trim spaces
while (*payload == ' ')
    payload++;

// For variant 4.1: payload should be 5 digits possibly;
we'll reverse the digit characters
char digits[16];
memset(digits, 0, sizeof(digits));
// copy digits only (up to 15 chars)
int di = 0;
for (char *p = payload; *p != '\0' && di <
(int)sizeof(digits) - 1; ++p)
{
    if (*p >= '0' && *p <= '9')
    {
        digits[di++] = *p;
    }
}
digits[di] = '\0';
if (di > 0)
{
    reverse_str(digits);
    // Send response WITHOUT variant prefix, terminated by
'\n'

    char resp[32];
    snprintf(resp, sizeof(resp), "%s\n", digits);
    uart_write_bytes(UART_PORT_NUM, resp, strlen(resp));
    ESP_LOGI(TAG, "RESP: %s", resp);
}
else
{
    ESP_LOGW(TAG, "Payload contained no digits, ignoring");
}
}
else
{
    // ignore other lines
    ESP_LOGI(TAG, "Not our variant -> ignored");
}

// reset line buffer
line_pos = 0;
memset(line, 0, sizeof(line));
}
}
// small delay to yield
vTaskDelay(pdMS_TO_TICKS(10));
}
}

```

Висновок: під час виконання лабораторної роботи я опанував принципи обміну даними через інтерфейс UART на мікроконтролері ESP32, використовуючи можливості багатозадачності FreeRTOS. Було програмно реалізовано алгоритм варіанта 4.1, який забезпечує налаштування параметрів зв'язку (1200 бод, контроль парності) та коректну обробку вхідних даних шляхом інвертування отриманого цифрового рядка. Тестування системи підтвердило надійність передачі та правильність логіки формування відповідей у режимі Loopback.